

Parallel LBM Solver Optimization Report

Author: Hang Li

Keywords: LBM, CFD, HPC, MPI, OpenMP, SIMD, Parallel Computing,
Domain Decomposition, Performance Optimization, Strong Scaling, Weak Scaling, C++

Date: 2025

Contents

1. Introduction	1
1.1. Introduction of LBM	1
1.1.1. Lattice Structure and Distribution Functions	1
1.1.2. Recovery of Macroscopic Quantities	1
1.1.3. Algorithmic Workflow of LBM	2
1.2. Overall of Parallelism Optimization	3
1.2.1. Multi-Level Parallelism Strategy	3
1.2.2. MPI-Based Domain Decomposition	3
1.2.3. OpenMP-Based Thread Parallelism	4
1.2.4. SIMD Vectorization via SoA Data Layout	4
1.2.5. Integrated Execution Flow	4
2. Process hierarchy optimization – MPI	5
2.1. MPI Parallelization Strategy	5
2.1.1. Domain Decomposition Strategy	5
2.1.2. Communication via Messenger Abstraction	6
2.2. Code Structure of the Two Main Functional Phases	6
2.2.1. Initialization Phase	7
2.2.2. Calculation Phase	7
2.3. Key MPI Code Components	7
2.3.1. MPI Initialization and Cartesian Topology	8
2.3.2. Domain Decomposition via Cartesian Coordinates	8
2.3.3. Non-Blocking MPI Communication	8
2.3.4. Synchronization of Communication Phase	9
2.3.5. MPI Timing and Performance Measurement	9
2.4. Performance Profiling Results	9
2.4.1. Strong Scaling Results	9
2.4.2. Breakdown of Initialization and Computation Phases for strong scaling	10
2.4.3. Weak Scaling Results	12
2.4.4. Further Analysis of Weak Scaling Behavior	13
3. Loop hierarchy optimization – OpenMp	15
3.1. Targeted Parallel Regions	15
3.1.1. Parallelization of the Streaming Step	15
3.1.2. Parallelization of the Collision Step	16
3.1.3. Parallelization of Validation and Reduction Loops	16
3.1.4. Why Communication step was not parallelized with OpenMP	17
3.2. Thread Safety and Data Integrity	18

3.3. Performance Impact	18
4. Data structure hierarchy optimization	21
4.1. SIMD Optimization and Data Layout	21
4.1.1. Compiler Auto-Vectorization Strategy	21
4.1.2. Data Layout Transformation: AoS vs. SoA	21
4.1.3. Illustrative Example: AoS to SoA Transformation	22
4.2. Performance Profiling Results	23
4.2.1. Compiler Vectorization Analysis	23
4.2.2. Execution Time Analysis	23
4.2.3. Speedup Analysis	24
4.3. Potential Performance Improvements	24
4.3.1. Eliminating Pointer Chasing in <code>stream()</code>	24
4.3.2. Removing Conditionals from Hot Loops	25
4.3.3. Memory Alignment	26
4.3.4. Ping-Pong Buffering	26
 I. Appendix	 27
.1. Final Version of Code:	28
 Bibliography	 31

1. Introduction

1.1. Introduction of LBM

The Lattice Boltzmann Method (LBM) is a mesoscopic numerical method that models fluid flow through the evolution of particle distribution functions on a discrete lattice. Instead of directly solving the macroscopic Navier–Stokes equations, LBM operates on distribution functions associated with a finite set of discrete velocity directions.

1.1.1. Lattice Structure and Distribution Functions

Figure 1.1 illustrates a typical lattice node and the discrete velocity directions used in LBM. At each lattice location $\mathbf{x}$, several distribution functions $f_i(\mathbf{x}, t)$ are defined, where the index i denotes a specific direction in the lattice.

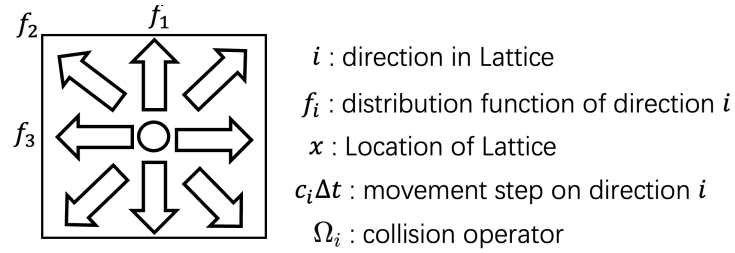


Figure 1.1.: Discrete lattice directions and corresponding distribution functions f_i at a lattice node.

Each distribution function f_i represents the amount of fluid particles moving along direction i . The discrete velocity vector $\mathbf{c}_i$ determines both the direction and the distance that particles travel during one time step Δt , resulting in a displacement $\mathbf{c}_i \Delta t$.

The evolution of f_i is governed by two physical processes:

- **Streaming**, which transports particle distributions to neighboring lattice nodes along direction i .
- **Collision**, which redistributes the distributions locally at each lattice node according to a collision operator Ω_i .

1.1.2. Recovery of Macroscopic Quantities

Although LBM operates on distribution functions, macroscopic flow variables are obtained through moment summations of f_i . The fluid density ρ and velocity $\mathbf{u}$ at each lattice node are computed as

$$\rho = \sum_i f_i, \quad (1.1)$$

$$\mathbf{u} = \frac{1}{\rho} \sum_i \mathbf{c}_i f_i. \quad (1.2)$$

These macroscopic quantities provide the physical interpretation of the lattice distributions and serve as inputs for constructing the equilibrium distribution function f_i^{eq} .

1.1.3. Algorithmic Workflow of LBM

The complete time-stepping procedure of LBM is summarized in Figure 1.2. The algorithm consists of a sequence of initialization, streaming, macroscopic update, and collision steps.

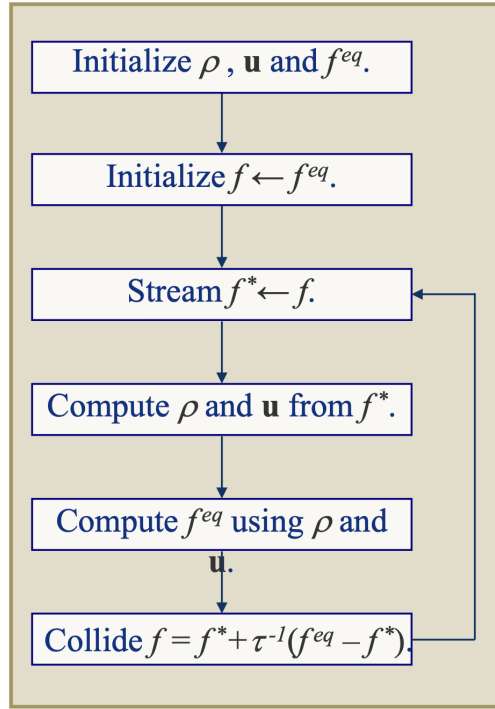


Figure 1.2.: Algorithmic flowchart of the Lattice Boltzmann Method.

At the beginning of the simulation, the macroscopic variables ρ and $\mathbf{u}$ are initialized, and the corresponding equilibrium distributions f_i^{eq} are computed. The distribution functions are then initialized as $f_i \leftarrow f_i^{eq}$.

During each time step, the following operations are performed:

1. **Streaming step:** the distribution functions propagate along their lattice directions,

$$f_i^*(\mathbf{x}, t + \Delta t) = f_i(\mathbf{x} - \mathbf{c}_i \Delta t, t). \quad (1.3)$$

2. **Macroscopic update:** density ρ and velocity $\mathbf{u}$ are recomputed from the streamed distributions f_i^* .
3. **Equilibrium reconstruction:** the equilibrium distribution f_i^{eq} is updated using the new macroscopic variables.

4. **Collision step:** the distributions relax towards equilibrium according to the BGK collision model,

$$f_i = f_i^* + \tau^{-1} (f_i^{eq} - f_i^*), \quad (1.4)$$

where τ is the relaxation time controlling the fluid viscosity.

This explicit streaming–collision structure, together with strictly local data dependencies, makes LBM particularly suitable for parallel computing architectures.

1.2. Overall of Parallelism Optimization

To fully exploit modern high-performance computing architectures, the Lattice Boltzmann Method (LBM) solver is optimized using a hierarchical parallelization strategy. The implementation combines three complementary levels of parallelism: MPI-based process-level parallelism, OpenMP-based thread-level parallelism, and SIMD vectorization enabled by a Structure-of-Arrays (SoA) data layout. This multi-layer design allows the solver to scale efficiently from single cores to distributed-memory systems.

1.2.1. Multi-Level Parallelism Strategy

The overall parallelization follows a natural hardware hierarchy. At the coarsest level, the computational domain is decomposed across multiple MPI processes. Within each process, OpenMP is used to parallelize loops over lattice nodes. Finally, at the innermost level, SIMD vectorization is exploited by ensuring contiguous memory access patterns through SoA data structures. This hierarchy can be summarized as:

- **MPI (process-level parallelism):** distributed-memory domain decomposition across processes.
- **OpenMP (thread-level parallelism):** shared-memory parallelism within each MPI process.
- **SIMD vectorization (instruction-level parallelism):** data-level parallelism inside computational kernels.

1.2.2. MPI-Based Domain Decomposition

MPI is responsible for large-scale parallelism by splitting the global lattice domain into subdomains, each handled by one MPI process. A Cartesian topology is constructed to reflect the geometric structure of the lattice, enabling structured neighbor communication. During the streaming phase, distribution functions located at subdomain boundaries are exchanged between neighboring MPI processes.

To reduce synchronization overhead, non-blocking point-to-point communication is employed. This design allows communication to overlap with computation and ensures scalability when increasing the number of processes. MPI communication is confined to well-defined phases of the time-stepping loop, preserving the locality of the collision operation.

1.2.3. OpenMP-Based Thread Parallelism

Inside each MPI process, OpenMP is used to parallelize loops over lattice nodes. The streaming, collision, and reporting routines are all parallelized using `#pragma omp parallel` for directives. Each thread operates on a distinct set of lattice nodes, avoiding data races by construction.

Static scheduling is employed to improve cache locality and ensure balanced workload distribution. Reduction clauses are used where global quantities, such as density sums, must be accumulated. This thread-level parallelism efficiently utilizes multi-core CPUs and complements the coarser MPI parallelism.

In the final implementation, OpenMP is applied to compute-intensive loops such as streaming, collision, and validation, while MPI communication remains outside OpenMP regions to ensure thread safety and avoid unnecessary synchronization overhead.

1.2.4. SIMD Vectorization via SoA Data Layout

At the finest granularity, SIMD vectorization is achieved through a Structure-of-Arrays (SoA) data layout. Distribution functions are stored in flattened one-dimensional arrays, where values corresponding to different directions of the same node are placed contiguously in memory.

This memory layout enables the compiler to automatically vectorize inner loops that iterate over lattice directions. Continuous memory access patterns reduce cache misses and allow efficient use of SIMD instruction sets such as AVX. Compiler optimization flags are explicitly enabled to encourage aggressive vectorization, and vectorization reports are used to verify the effectiveness of this approach.

1.2.5. Integrated Execution Flow

During one LBM time step, the three levels of parallelism interact as follows:

1. Each MPI process advances its local subdomain independently.
2. Within a process, OpenMP threads parallelize node-wise operations such as streaming and collision.
3. Inside these threaded loops, SIMD instructions operate on multiple distribution values simultaneously.

This layered execution model ensures that computation is efficiently mapped to the underlying hardware at all scales, from distributed nodes down to vector registers.

2. Process hierarchy optimization – MPI

2.1. MPI Parallelization Strategy

At the highest level of parallelism, the LBM solver employs MPI to achieve distributed-memory parallelization through domain decomposition. The global computational lattice is partitioned into multiple subdomains, each assigned to an individual MPI process. This strategy enables scalable execution across multiple nodes while preserving the locality of lattice-based computations.

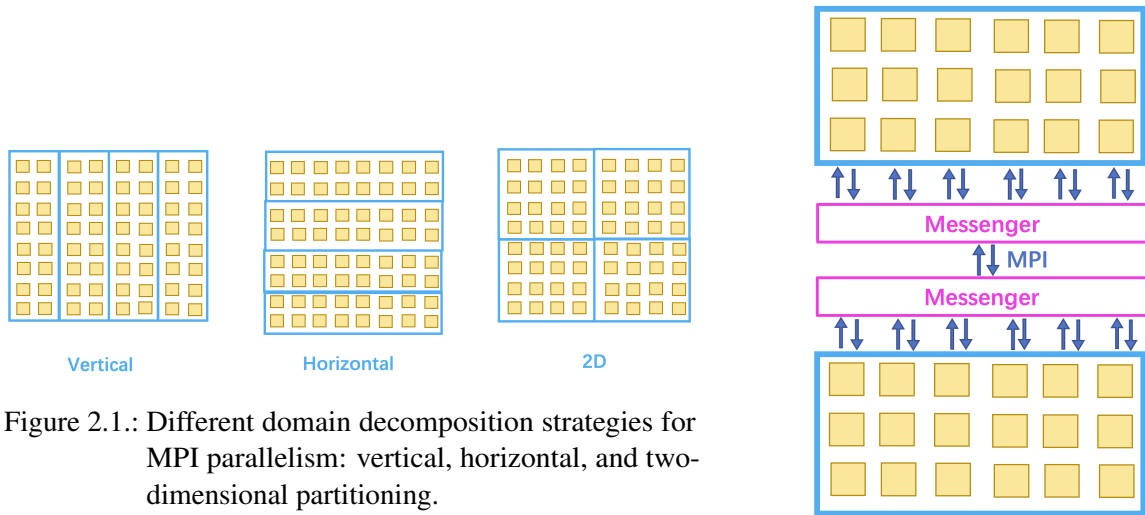


Figure 2.1.: Different domain decomposition strategies for MPI parallelism: vertical, horizontal, and two-dimensional partitioning.

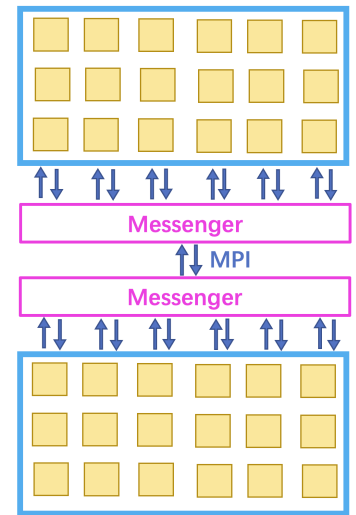


Figure 2.2.: Messenger-based communication abstraction.

2.1.1. Domain Decomposition Strategy

Figure 2.1 illustrates three possible domain decomposition strategies. In the vertical and horizontal decompositions, the global lattice is split along a single spatial direction, resulting in strip-shaped subdomains. While simple to implement, these strategies may lead to load imbalance or increased communication overhead for certain domain shapes.

To address this limitation, a two-dimensional decomposition is adopted, in which the lattice is partitioned along both spatial directions. This approach produces more compact subdomains, reduces the surface-to-volume ratio, and minimizes the amount of boundary data that must be exchanged between MPI processes. As a result, two-dimensional decomposition provides better scalability for large problem sizes.

2.1.2. Communication via Messenger Abstraction

As shown in Figure 2.2, communication between neighboring subdomains is handled through a dedicated *Messenger* abstraction. Each Messenger represents a logical communication channel responsible for exchanging distribution function values located at subdomain boundaries.

During the streaming phase of the LBM time step, particle distributions that propagate across subdomain interfaces are collected and transferred via MPI. The Messenger layer decouples the communication logic from the core numerical kernels, allowing the streaming and collision steps to remain largely unchanged in the presence of MPI parallelism.

Non-blocking MPI communication is employed to exchange boundary data between adjacent processes, enabling overlap between communication and computation. This design preserves the locality of the collision operation while ensuring consistent data propagation across subdomain boundaries.

2.2. Code Structure of the Two Main Functional Phases

The MPI-parallel LBM implementation is organized around two clearly separated functional phases: an *initialization phase* and a *calculation phase*. This separation is not only conceptual, but also explicitly reflected in the code structure and execution flow, as illustrated in Figure 2.3.

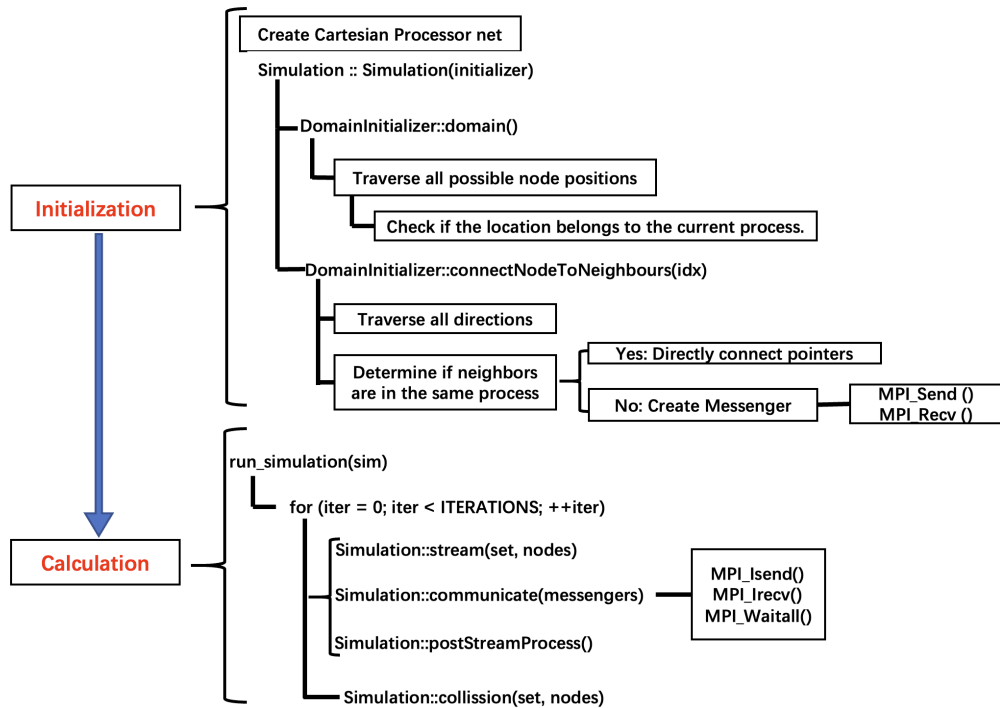


Figure 2.3.: Code structure and execution flow of the MPI-parallel LBM solver, highlighting the separation between initialization and calculation phases.

2.2.1. Initialization Phase

The initialization phase is responsible for constructing a distributed lattice representation and establishing all data dependencies required for MPI-parallel execution. This phase is executed once at the beginning of the simulation and includes both domain decomposition and communication setup.

First, a Cartesian processor topology is created using MPI, which defines the logical neighborhood relationships between MPI processes. This topology serves as the basis for consistent domain decomposition and neighbor identification.

Next, the `Simulation` object is constructed, which internally triggers the domain initialization through `DomainInitializer::domain()`. During this process, all possible lattice node positions are traversed, and each node is assigned to a specific MPI process according to the chosen decomposition strategy. Only nodes belonging to the current process are instantiated locally.

After node creation, neighbor connections are established in

`DomainInitializer::connectNodeToNeighbours()`. For each node and each lattice direction, the code determines whether the corresponding neighbor lies within the same MPI process or in a remote process. If the neighbor is local, distribution pointers are directly connected, enabling zero-overhead data exchange within a process. If the neighbor is remote, a `Messenger` object is created to represent a communication channel, and the necessary MPI metadata is stored for later use.

As a result of the initialization phase, all communication patterns are fully determined and encoded in data structures. No further topology analysis or neighbor discovery is required during the time-stepping loop.

2.2.2. Calculation Phase

The calculation phase corresponds to the actual time integration of the LBM solver and is executed repeatedly for a prescribed number of iterations. Its structure is deliberately simple and regular, relying entirely on the data structures prepared during initialization.

Each iteration consists of a fixed sequence of operations. First, the streaming step propagates distribution values along lattice directions. Due to the pointer-based neighbor connections established earlier, this operation is implemented uniformly for both local and remote neighbors. Local updates write directly into neighbor nodes, while remote updates write into the source buffers of associated `Messenger` objects.

Following streaming, the `communicate()` function is invoked. At this stage, all data stored in messengers is exchanged between MPI processes using non-blocking MPI communication routines. Received data is then written into the appropriate local lattice nodes, completing the propagation of distributions across subdomain boundaries.

After communication, a post-stream processing step is executed to apply boundary conditions or other local corrections. Finally, the collision step updates the distribution functions according to the BGK collision operator, using only local data.

2.3. Key MPI Code Components

This section highlights the key MPI-related code components that implement the distributed-memory parallelization strategy described earlier. Only the essential code fragments are presented here, focusing on structure and design decisions rather than low-level implementation details. For each

fragment, the corresponding source file and location are explicitly indicated to facilitate code inspection.

2.3.1. MPI Initialization and Cartesian Topology

Location: `src/main.cpp` (lines 110–142)

At program startup, MPI is initialized and a two-dimensional Cartesian topology is created. This topology defines the logical neighborhood relationships between MPI processes and serves as the basis for domain decomposition.

```

1  MPI_Init(&argc , &argv );
2
3  int dims[2] = {0, 0};
4  int periods[2] = {1, 1}; // periodic boundaries
5  MPI_Dims_create(p, 2, dims);
6
7  MPIComm cart_comm;
8  MPI_Cart_create(MPI_COMM_WORLD, 2, dims, periods, 1, &cart_comm);
9
10 int coords[2];
11 MPI_Cart_coords(cart_comm, rank, 2, coords);

```

This approach allows MPI to automatically determine an efficient 2D process grid, reducing communication distance and simplifying neighbor identification.

2.3.2. Domain Decomposition via Cartesian Coordinates

Location: `src/Domains/DomainInitializer.cpp` (lines 398–415)

During domain initialization, each lattice node is assigned to an MPI process based on its spatial position and the Cartesian topology.

```

1  int px = d_cart_dims[0];
2  int py = d_cart_dims[1];
3
4  int block_x = (position[0] * px) / d_domain_size[0];
5  int block_y = (position[1] * py) / d_domain_size[1];
6
7  int coords[2] = {block_x, block_y};
8  int rank;
9  MPI_Cart_rank(d_cart_comm, coords, &rank);

```

Using Cartesian coordinates avoids manual rank computations and ensures a consistent mapping between spatial locality and process topology.

2.3.3. Non-Blocking MPI Communication

Location: `src/LBM/Simulation.cpp` (lines 196–227)

Inter-process communication occurs during the streaming phase via non-blocking MPI operations. Each boundary-crossing distribution is sent asynchronously to its target process.

```

1  MPI_Isend(tag_data, 2, MPI_UNSIGNED_LONG,
2           static_cast<int>(messenger.d_p), 0, comm, &req);
3

```

```

4 MPI_Isend(&messenger.d_src, 1, MPI_DOUBLE,
5          static_cast<int>(messenger.d_p), 1, comm, &req);
6
7 send_requests.push_back(req);

```

Non-blocking communication enables overlap between communication and computation and removes the global synchronization present in the original BSP-based implementation.

2.3.4. Synchronization of Communication Phase

Location: src/LBM/Simulation.cpp (lines 231–237)

To ensure correctness when combining non-blocking sends with blocking receives, all outstanding communication requests are synchronized explicitly.

```

1 if (!send_requests.empty())
2 {
3     std::vector<MPI_Status> statuses(send_requests.size());
4     MPI_Waitall(send_requests.size(),
5                send_requests.data(),
6                statuses.data());
7 }
8
9 MPI_Barrier(comm);

```

This synchronization guarantees that all messages have been issued before the receive phase begins, preventing deadlocks while maintaining a clear communication boundary within each iteration.

2.3.5. MPI Timing and Performance Measurement

Location: src/LBM/Simulation.cpp (lines 35–65)

MPI-provided wall-clock timing is used to measure the cost of each major phase in the simulation loop.

```

1 double t_start = MPI_Wtime();
2 stream(d_domain->set, d_domain->nodes);
3 d_stream_time += MPI_Wtime() - t_start;
4
5 t_start = MPI_Wtime();
6 communicate(d_domain->messengers);
7 d_communicate_time += MPI_Wtime() - t_start;

```

This fine-grained timing enables quantitative analysis of computation and communication costs, which is essential for evaluating scalability.

2.4. Performance Profiling Results

2.4.1. Strong Scaling Results

Figure 2.4 shows the strong scaling results for a fixed problem size of 200×200 lattice nodes and 1000 time iterations. The horizontal axis denotes the number of MPI processes, while the vertical axis shows the total execution time on a logarithmic scale. An ideal strong scaling curve with

slope -1 is included as a reference. Three domain decomposition strategies are compared: vertical, horizontal, and two-dimensional (2D) decomposition.

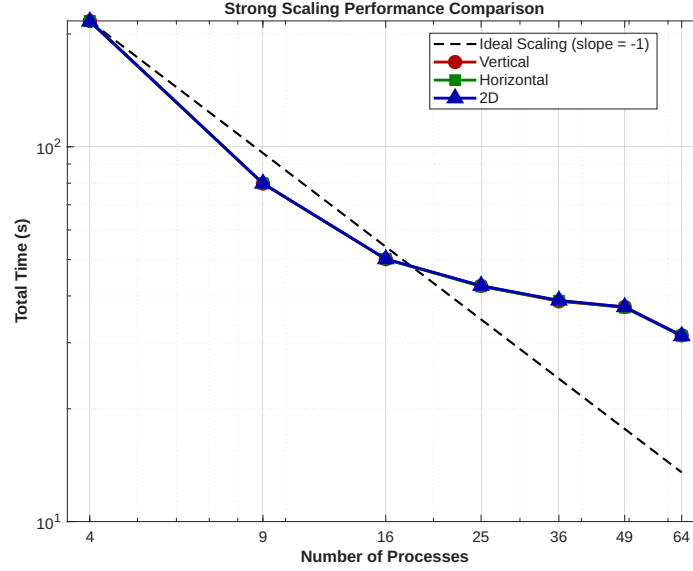


Figure 2.4.: Strong scaling performance comparison for different domain decomposition strategies.

Overall, the measured runtimes of the three decomposition strategies are very close to each other across the entire range of process counts. No significant performance difference can be observed between vertical, horizontal, and 2D decomposition for this problem size, indicating that all three strategies lead to comparable computational and communication costs in this configuration.

At low process counts, the solver exhibits good strong scaling behavior, with the total runtime decreasing nearly proportionally to the number of processes. As the process count increases further, the scaling curve gradually flattens and deviates from the ideal scaling trend. This saturation behavior suggests that overheads beyond pure computation begin to dominate the runtime.

The observed flattening motivates a more detailed investigation of the underlying performance bottlenecks. In the following section, the contribution of communication and synchronization overheads is analyzed in order to explain the deviation from ideal strong scaling at higher process counts.

2.4.2. Breakdown of Initialization and Computation Phases for strong scaling

To further explain the strong scaling behavior observed in the previous section, Figure 2.8 presents a decomposed view of the total runtime. The strong scaling results are separated into the initialization phase and the computation phase, together with the relative contribution of the initialization time. All measurements correspond to the same configuration as before, i.e. a fixed problem size of 200×200 lattice nodes and 1000 iterations.

2. Process hierarchy optimization – MPI

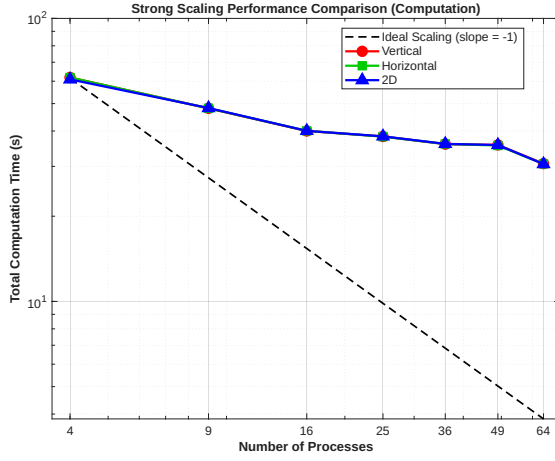


Figure 2.5.: Strong scaling of computation time.

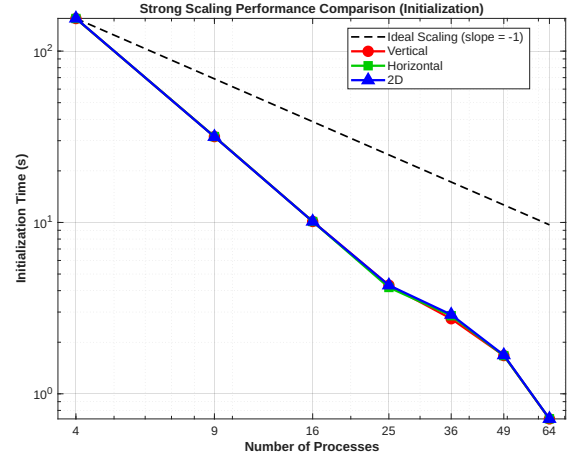


Figure 2.6.: Strong scaling of initialization time.

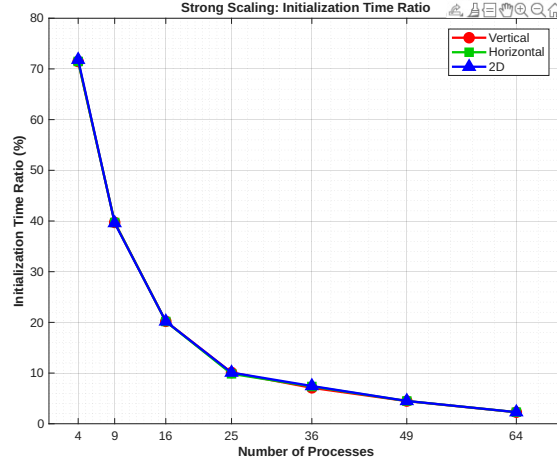


Figure 2.7.: Ratio of initialization time to total runtime.

Figure 2.8.: Breakdown of strong scaling results into computation and initialization phases, and the corresponding initialization time ratio.

From the computation-time plot, it can be seen that increasing the number of processes continues to reduce the computation time, indicating that process-level parallelism remains effective for the core numerical workload. Although the scaling efficiency gradually decreases, computation still benefits from higher process counts.

In contrast, the initialization-time plot shows a much stronger dependence on the number of processes. At low process counts, the initialization time decreases rapidly as the workload is distributed across processes. However, this benefit diminishes quickly as the process count increases, and the initialization time approaches a lower bound.

This trend is further highlighted in the ratio plot. As the number of processes grows, the fraction of time spent in initialization drops sharply and then stabilizes at a low level. The marginal benefit of adding more processes to the initialization phase becomes negligible at high process counts.

These observations explain the flattening behavior in the strong scaling results shown previously.

While computation continues to scale, the limited scalability of the initialization phase reduces the overall marginal benefit of additional processes, causing the total runtime to deviate from ideal strong scaling at high process counts.

2.4.3. Weak Scaling Results

Figure 2.11 presents the weak scaling results of the LBM solver. In this experiment, the problem size is scaled proportionally with the number of MPI processes, using a fixed local problem size of 25×25 lattice nodes per process and 1000 iterations. The total runtime and the computation-only runtime are shown separately, and an ideal weak scaling curve corresponding to constant runtime is included as a reference.

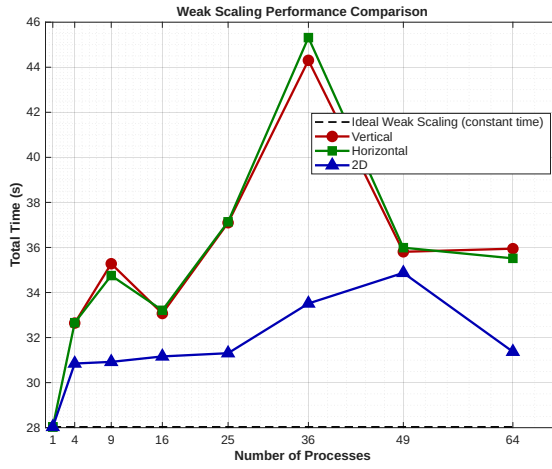


Figure 2.9.: Weak scaling performance (total runtime).

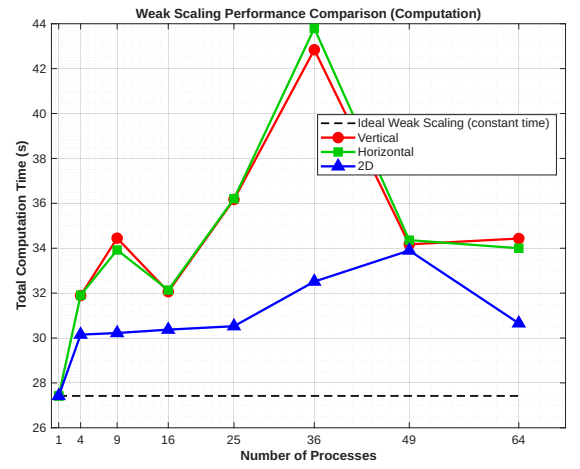


Figure 2.10.: Weak scaling performance (computation phase only).

Figure 2.11.: Weak scaling results for a fixed local problem size of 25×25 per process.

From the total runtime plot, it can be observed that the vertical and horizontal domain decomposition strategies exhibit almost identical behavior across the entire range of process counts. Both show noticeable fluctuations around the ideal weak scaling line, indicating sensitivity to communication patterns and process placement. In contrast, the two-dimensional (2D) decomposition produces a more stable runtime and consistently outperforms the other two strategies.

The computation-only plot shows a very similar trend to the total runtime. This indicates that the observed weak scaling behavior is primarily governed by the computation phase rather than by initialization overhead. Due to the small local problem size assigned to each process, the cost of the initialization phase becomes negligible and does not contribute significantly to the overall runtime. These observations suggest that, under weak scaling conditions, performance differences between decomposition strategies are dominated by communication and runtime variability rather than initialization costs. In particular, the improved stability of the 2D decomposition motivates a more detailed analysis of communication behavior in the following section using Scalasca.

2.4.4. Further Analysis of Weak Scaling Behavior

To further analyze the weak scaling behavior discussed in the previous section, we examine two complementary aspects: a function-level runtime breakdown obtained with Scalasca, and the weak scaling efficiency. The two analyses are presented separately and address different questions regarding performance behavior.

Function-Level Analysis with Scalasca

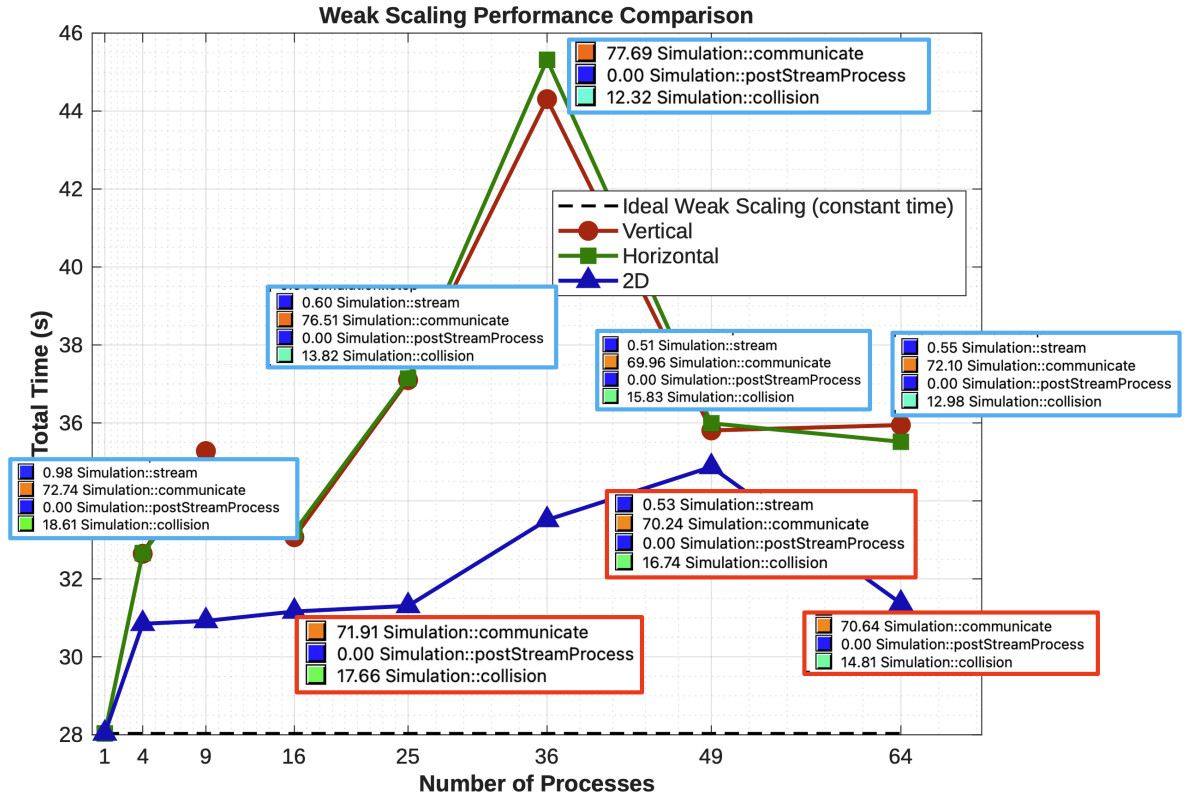


Figure 2.12.: Scalasca function-level time percentage for the weak scaling experiment, focusing on the computation phase.

Figure 2.12 shows the Scalasca analysis results for the weak scaling experiment, focusing on the computation phase. The figure reports the relative time percentages spent in different functions, aggregated across MPI ranks.

The most prominent observation is that the `Simulation::communicate` function accounts for the largest fraction of the runtime in all domain decomposition strategies. This confirms that inter-process communication is the dominant cost under weak scaling conditions. Among the three strategies, the two-dimensional (2D) decomposition consistently exhibits a smaller communication percentage compared to the vertical and horizontal decompositions. This reduced communication share provides a plausible explanation for the superior weak scaling performance of the 2D strategy observed previously.

Scalasca further indicates that the workload is well balanced across MPI ranks for all decomposition strategies. No significant load imbalance between ranks is detected, suggesting that uneven work distribution is not the primary cause of the observed runtime fluctuations. As a result, it remains difficult to attribute the performance variability to algorithmic factors alone. A plausible explanation is that hardware-related effects, such as network contention or process placement, contribute to the observed fluctuations.

Weak Scaling Efficiency

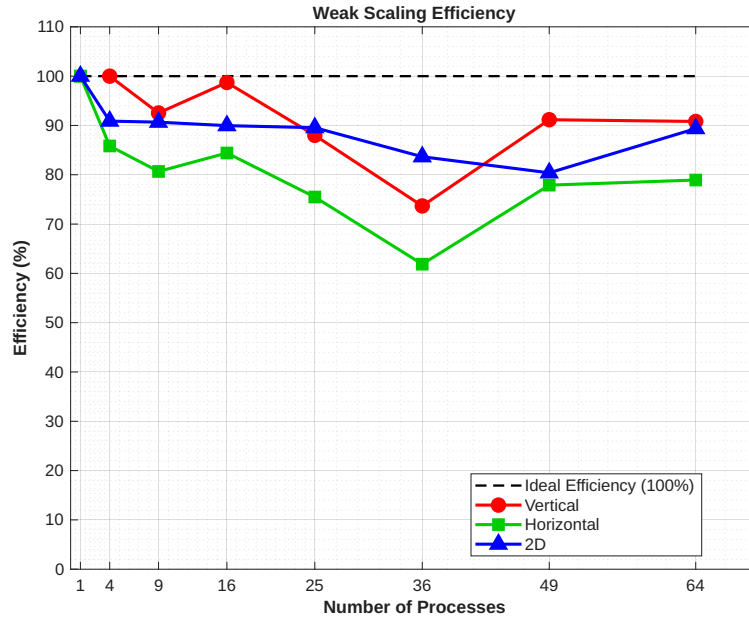


Figure 2.13.: Weak scaling efficiency as a function of the number of MPI processes.

Figure 2.13 presents the weak scaling efficiency, defined as

$$\eta_{\text{weak}}(P) = \frac{T(1)}{T(P)}, \quad (2.1)$$

where $T(P)$ denotes the total runtime using P MPI processes. An ideal weak scaling behavior corresponds to an efficiency of 100%.

Both the vertical and 2D decomposition strategies demonstrate good weak scaling efficiency across most process counts, with efficiency values largely remaining above 80%. This indicates that the solver maintains a reasonable level of parallel efficiency even as the global problem size increases. An interesting observation is the presence of a consistent efficiency gap between the vertical and horizontal decompositions. The vertical strategy systematically achieves higher efficiency than the horizontal one. Since algorithmic structure and workload balance are comparable, this regular gap is likely caused by hardware-related factors, such as communication topology, network routing, or node-level resource contention.

Overall, the efficiency analysis supports the conclusions drawn from the Scalasca results: communication behavior plays a central role in weak scaling performance, while the remaining performance variations are likely influenced by hardware effects rather than by the decomposition strategy itself.

3. Loop hierarchy optimization – OpenMp

While MPI is used to exploit distributed-memory parallelism across processes, significant computational work remains within each MPI subdomain. To efficiently utilize the multiple CPU cores available on each compute node, OpenMP is used to introduce shared-memory thread-level parallelism inside each MPI process.

3.1. Targeted Parallel Regions

The Lattice Boltzmann Method naturally exposes loop-level parallelism, as most operations are performed independently for each lattice node. Based on profiling results, OpenMP parallelization was applied to the most time-consuming loops inside the calculation phase, namely:

- the streaming loop,
- the collision loop,
- and validation and reduction-based reporting loops.

Each of these loops iterates over local lattice nodes and operates exclusively on node-local data structures, making them well suited for OpenMP parallelization.

3.1.1. Parallelization of the Streaming Step

The streaming operation propagates distribution functions from neighboring nodes into the current node. After refactoring the streaming kernel into a pull-based formulation with index-based neighbor access, each node update becomes fully independent.

As a result, the outer loop over lattice nodes is parallelized using OpenMP:

```
1  #pragma omp parallel for reduction(+:null_neighbour_count) schedule(static)
2  for (size_t idx = 0; idx < nodes.size(); ++idx) {
3      if (nodes[idx].distributions == nullptr) {
4          continue;
5      }
6      for (size_t dir = 0; dir < nDirections; ++dir) {
7          if (nodes[idx].distributions[dir].neighbour != nullptr) {
8              *nodes[idx].distributions[dir].neighbour = nodes[idx].distributions[
9                  dir].value;
10             }
11             else {
12                 null_neighbour_count++;
13             }
14 }
```

Listing 3.1: Destination: /src/LBM/Simulation.cpp line 66

Static scheduling is chosen to ensure predictable memory access patterns and improved cache locality. Since each thread writes only to its own node-local buffers, no synchronization or atomic operations are required.

3.1.2. Parallelization of the Collision Step

The collision step performs local relaxation of distribution functions toward equilibrium. Similar to streaming, collision computations are strictly local to each node and have no data dependencies between nodes.

The collision loop is parallelized using the same OpenMP strategy:

```

1  #pragma omp parallel for schedule(static)
2  for (size_t i = 0; i < nodes.size(); ++i) {
3      Node &node = nodes[i];
4
5      if (node.distributions == nullptr) continue;
6
7      size_t nDirections = set->nDirections;
8      for (size_t dir = 0; dir < nDirections; ++dir) {
9          if (node.distributions[dir].nextValue >= 0.0) {
10             node.distributions[dir].value = node.distributions[dir].nextValue;
11          }
12          node.distributions[dir].nextValue = -1;
13      }
14
15      auto node_equilibrium = equilibrium(set, node);
16      for (size_t dir = 0; dir < nDirections; ++dir)
17          node.distributions[dir].value = node.distributions[dir].value - omega *
18              (node.distributions[dir].value - node_equilibrium[dir]);
19
20      delete[] node_equilibrium;
21  }
```

Listing 3.2: Destination: /src/LBM/Simulation.cpp line 91

This approach allows collision computations to scale efficiently with the number of available CPU cores inside each MPI process.

3.1.3. Parallelization of Validation and Reduction Loops

In addition to the main simulation kernels, validation and reporting routines compute global quantities such as total density and velocity by iterating over all lattice nodes. These loops were also parallelized using OpenMP reductions:

```

1  #pragma omp parallel for reduction(+:current_density) schedule(static)
2  for (size_t i = 0; i < d_domain->nodes.size(); ++i) {
3      current_density += density(d_domain->set, d_domain->nodes[i]);
4  }
```

Listing 3.3: Destination: /src/LBM/Simulation.cpp line 356

```

1  #pragma omp parallel for reduction(+:total_density) schedule(static)
2  for (size_t idx = 0; idx < totalNodes; ++idx){
3      total_density += density(set, nodes[idx]);
4  }
```

```

4
5     double *node_velocity = velocity(set, nodes[idx]);
6     for (size_t dim = 0; dim < nDimensions; ++dim)
7         total_velocity[dim] += node_velocity[dim];
8
9     delete[] node_velocity;
10 }

```

Listing 3.4: Destination: /src/Reporting/reporting.cpp line 97

Reduction clauses ensure correctness while enabling efficient parallel accumulation of global quantities.

3.1.4. Why Communication step was not parallelized with OpenMP

The MPI communication phase was intentionally left outside OpenMP parallel regions. Although the communication step iterates over multiple messages, parallelizing it with OpenMP creates risks to disrupt correctness and performance.

First, MPI calls such as `MPI_Isend`, `MPI_Recv`, and `MPI_Iprobe` are not guaranteed to be thread-safe unless the MPI library is initialized with `MPI_Init_thread` and provides full `MPI_THREAD_MULTIPLE` support. Even when supported, `MPI_THREAD_MULTIPLE` often incurs significant internal locking overhead, which can degrade performance.

Second, the communication logic relies on shared data structures such as message buffers, request vectors, and receive queues. Parallel access to these structures would require explicit synchronization (e.g., critical sections or locks), increasing contention and potentially eliminating any benefit from parallel execution.

Finally, the communication phase is latency-bound rather than compute-bound. Profiling results show that communication time dominates total runtime at higher OpenMP thread counts, indicating that further parallelization of computation would not reduce communication cost. Parallelizing communication with OpenMP would therefore provide limited performance benefit while substantially increasing implementation complexity.

Table 3.1.: Performance breakdown for different OpenMP thread counts (MPI processes = 4, domain size = 80×80 , iterations = 1000).

OMP Threads	Total Time [s]	Communication Time [s]	Communication Share [%]
1	36.26	24.78	68.3
2	32.04	23.03	71.9
4	28.05	23.52	83.9
8	26.43	23.82	90.1

Table 3.1 shows that increasing the number of OpenMP threads significantly reduces computation time, while communication time remains nearly constant. As a result, the relative share of communication increases from approximately 68% with one OpenMP thread to over 90% with eight threads. This indicates that the application becomes communication-bound at higher thread counts, and further OpenMP parallelization of computation cannot substantially reduce overall runtime.

For these reasons, OpenMP is applied exclusively to compute-intensive and embarrassingly parallel loops (streaming, collision, and validation), while MPI communication remains sequential at the process level.

3.2. Thread Safety and Data Integrity

Correctness under OpenMP parallel execution is ensured by construction. Each OpenMP thread operates on a distinct subset of lattice nodes, and no two threads write to the same memory location during streaming or collision. Communication buffers and MPI calls are executed outside OpenMP regions, avoiding unsafe interaction between MPI and OpenMP.

As a result, no locks, critical sections, or atomic operations are required, preserving both performance and numerical correctness.

Communication was intentionally not parallelized using OpenMP. The communication phase performs MPI send and receive operations and updates shared ghost cells at subdomain boundaries. Parallelizing this phase with OpenMP would introduce potential race conditions on shared boundary data and require the MPI implementation to support `MPI_THREAD_MULTIPLE`, which is not guaranteed and often incurs significant overhead. Furthermore, communication performance is dominated by network latency rather than computation, meaning that OpenMP parallelization would not provide meaningful performance benefits while increasing the risk of data inconsistency.

3.3. Performance Impact

Performance measurements demonstrate that OpenMP parallelization significantly reduces the computation time of the collision phase and small reduce in streaming phase. As the number of OpenMP threads increases, collision time decreases nearly linearly until communication overhead becomes dominant.

Since MPI communication remains serial within each process, overall speedup is bounded by communication costs, which become increasingly dominant at higher thread counts. Nevertheless, OpenMP provides a clear performance benefit by accelerating compute-intensive kernels and improving overall node-level utilization.

Table 3.2.: Performance results for different OpenMP thread counts (MPI processes = 4, domain size = 80×80 , iterations = 1000).

OMP Threads	Total Comp. Time [s]	Stream Time [s]	Collision Time [s]	Comm. Time [s]
1	36.26	0.1176	11.1237	24.7766
2	32.04	0.1897	8.7741	23.0292
4	28.05	0.0998	4.3918	23.5228
8	26.43	0.1023	2.4377	23.8215

Table 3.2 summarizes the measured execution times used to generate the performance plots and highlights the increasing dominance of communication time as computational phases are accelerated. Figure 3.1 shows the total computation time as a function of the number of OpenMP threads. Increasing the thread count leads to a consistent reduction in total computation time, with diminishing returns beyond four threads. This behavior indicates effective parallelization of compute-intensive kernels, while overall scalability is limited by the communication phase, which remains serial. Figure 3.2 illustrates the execution time of the streaming step. The streaming phase shows limited scalability with increasing OpenMP threads, as it represents a relatively small fraction of the total runtime and involves memory-bound operations with irregular access patterns.

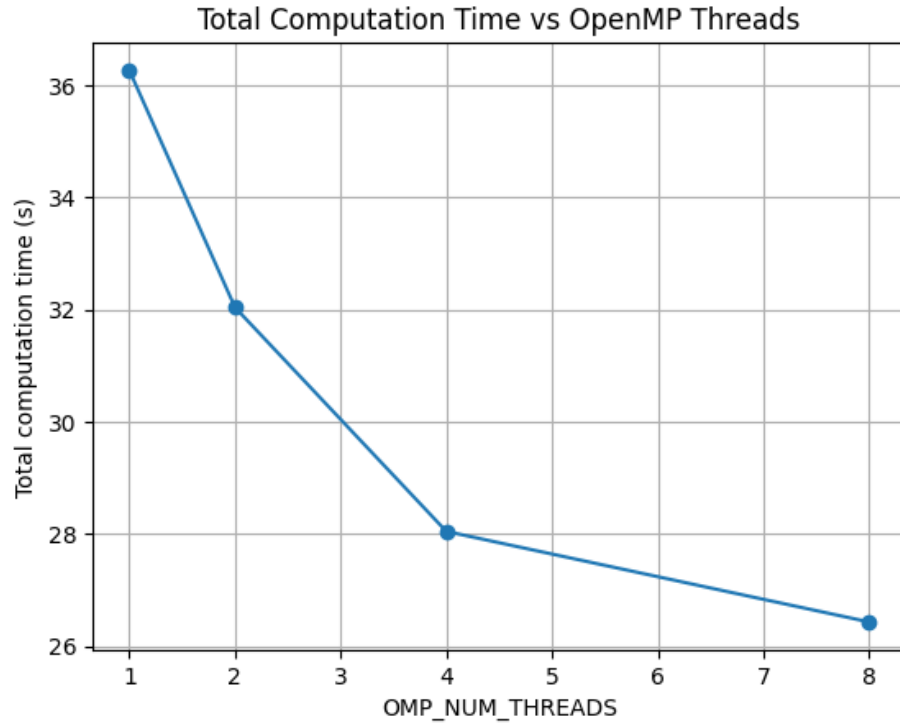


Figure 3.1.: Total computation time over different number of OpenMP threads

Figure 3.3 demonstrates strong scaling of the collision step. Collision time decreases significantly as the number of OpenMP threads increases, confirming that this phase is compute-bound and well suited for shared-memory parallelization.

Overall, the results indicate that OpenMP parallelization effectively accelerates compute-heavy phases of the LBM solver, particularly the collision step. However, as OpenMP thread count increases, communication time becomes the dominant cost, limiting further speedup. This confirms that future performance improvements would require communication optimizations rather than additional shared-memory parallelism.

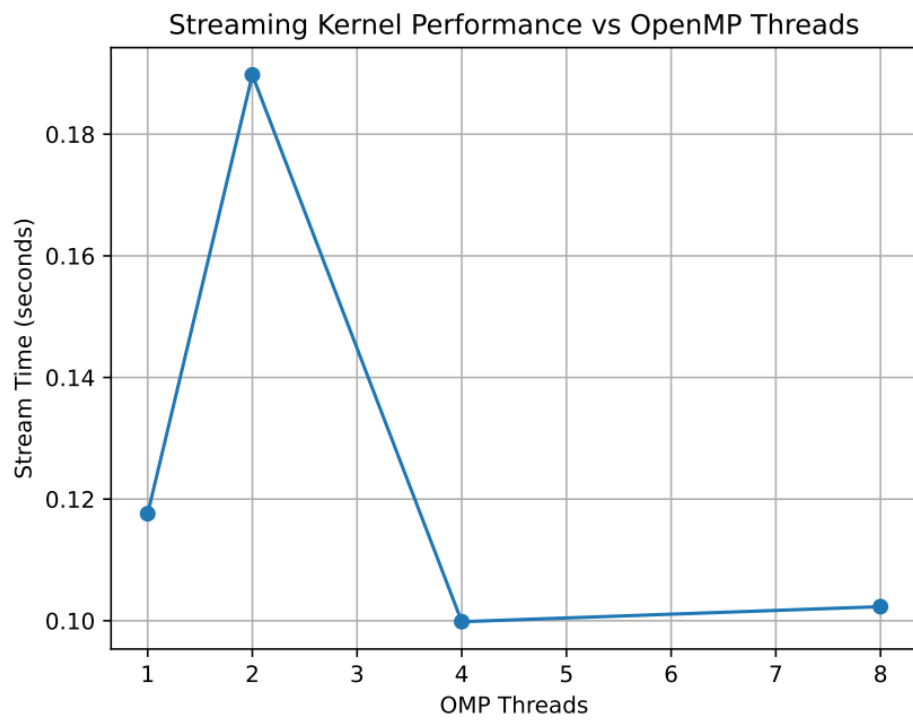


Figure 3.2.: Stream computation time over different number of OpenMP threads

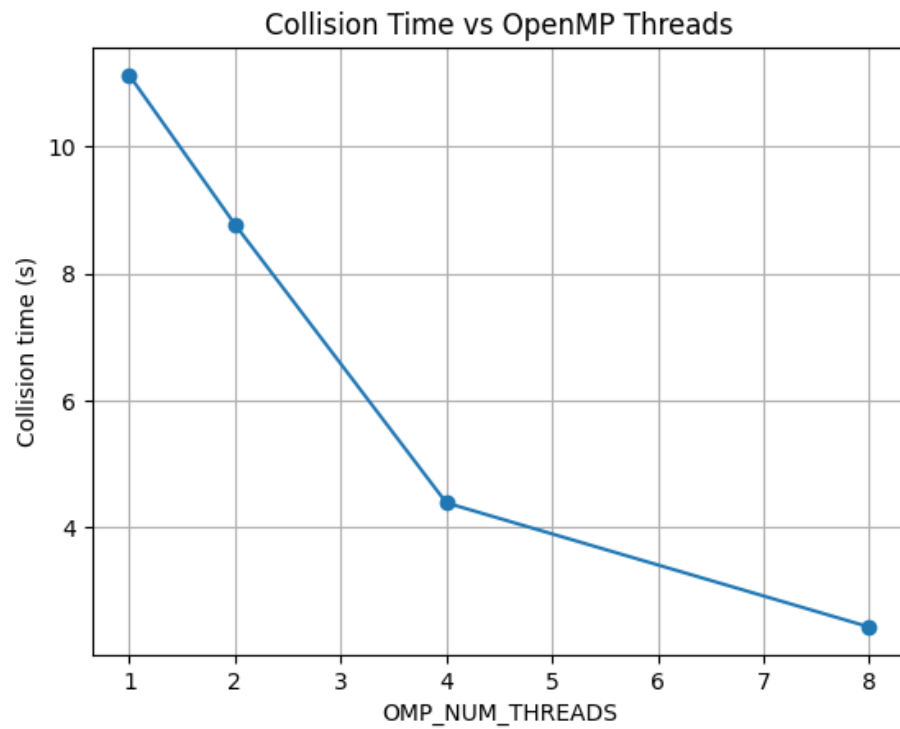


Figure 3.3.: Collision computation time over different number of OpenMP threads

4. Data structure hierarchy optimization

4.1. SIMD Optimization and Data Layout

4.1.1. Compiler Auto-Vectorization Strategy

Objective: Enable the compiler to generate wide SIMD instructions (AVX-512) for the performance-critical kernels, namely the `stream()` and `collide()` loops.

- **Baseline (Scalar build):**

- Compiler flags: `-O3 -fno-tree-vectorize -mno-avx`
- Vectorization and AVX extensions are explicitly disabled in order to establish a purely scalar reference baseline.
- This configuration allows accurate measurement of speedup attributable solely to SIMD optimizations.

- **Optimized (SIMD-enabled build):**

- Compiler flags: `-O3 -march=native -fopt-info-vec-optimized`
- `-march=native` enables architecture-specific instruction sets on the target platform (Intel Sapphire Rapids), including AVX-512 vector registers and fused multiply-add (FMA) operations.
- `-fopt-info-vec-optimized` generates a detailed vectorization report, identifying which loops were successfully vectorized and which failed, along with diagnostic reasons.

The intent of this strategy is to rely primarily on compiler auto-vectorization rather than manual intrinsics, thereby preserving code maintainability while still achieving high performance.

4.1.2. Data Layout Transformation: AoS vs. SoA

Goal: Maximize contiguous memory access and eliminate gather-heavy access patterns.

Array of Structures (AoS) – Original Layout.

- Data structure: `Node nodes[N]`, where each `Node` contains 9 double-precision distribution values.
- *Limitation:* Each lattice direction is interleaved across memory. Loading the same distribution component for multiple nodes requires strided or gather loads.
- This layout prevents efficient SIMD utilization and significantly degrades memory throughput.

Structure of Arrays (SoA) – Refactored Layout.

- Data structure: `std::vector<double> f[9]`, i.e., nine independent arrays, each of length N .
- *Benefit*: Each direction is stored contiguously in memory, enabling unit-stride vector loads and stores.
- This transformation is essential for achieving high utilization of wide SIMD registers such as AVX-512 (512-bit = 8 double-precision values per vector).

With this layout, the streaming and collision kernels can operate on eight lattice nodes simultaneously using aligned vector instructions.

4.1.3. Illustrative Example: AoS to SoA Transformation

To make the impact of the data layout transformation explicit, we provide a minimal code example illustrating the change from the original Array-of-Structures (AoS) layout to the refactored Structure-of-Arrays (SoA) layout.

AoS Layout (Original). Each lattice node stores all distribution values locally:

```
1 struct Node {
2     double f[9];
3 };
4
5 std::vector<Node> nodes(N);
6
7 // update distribution in direction d
8 for (size_t i = 0; i < N; ++i) {
9     nodes[i].f[d] *= omega;
10 }
```

In this layout, accesses to `nodes[i].f[d]` are strided in memory when iterating over `i`, which prevents efficient SIMD vectorization and results in gather-heavy memory access.

SoA Layout (Refactored). Distribution values are separated by direction and stored contiguously:

Defined Location: `src/Domains/Domain.h` and

`src/Domains/DomainInitializer.cpp` (lines 37–52)

```
1 std::vector<double> f[9];
2 for (int d = 0; d < 9; ++d)
3     f[d].resize(N);
4
5 // update distribution in direction d
6 for (size_t i = 0; i < N; ++i) {
7     f[d][i] *= omega;
8 }
```

Here, `f[d][i]` is accessed with unit stride over `i`, enabling contiguous loads and stores and exposing auto-vectorization opportunities to the compiler.

4.2. Performance Profiling Results

4.2.1. Compiler Vectorization Analysis

Status: Limited and fragmented vectorization. The compiler vectorization report (`vec_report.txt`) reveals two dominant blockers:

1. **Dynamic Memory Allocation in Hot Loops.** Repeated calls to operator `new` (originating from `std::vector` resizing and temporary object creation) inhibit vectorization entirely.
2. **Complex Control Flow.** Residual branching and indirect memory accesses in `stream()` prevent the compiler from forming wide vector loops.

4.2.2. Execution Time Analysis

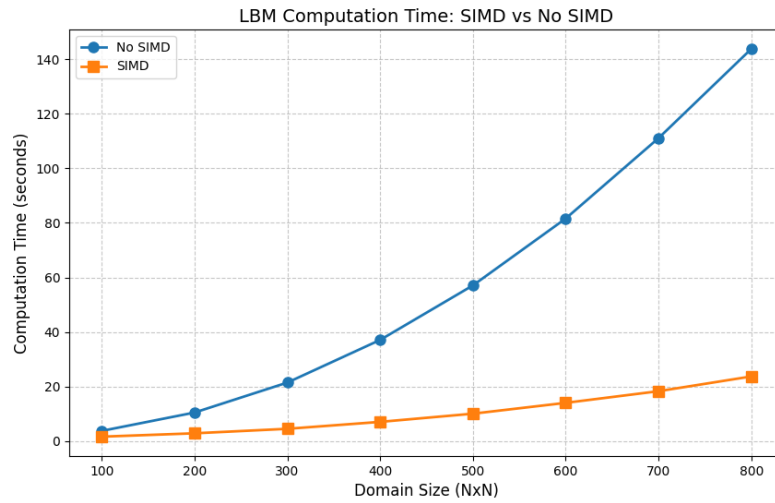


Figure 4.1.: Strong scaling: execution time comparison between scalar and SIMD-enabled builds

Figure 4.1 compares the execution time of the scalar and SIMD-enabled implementations as a function of the domain size. For both versions, execution time increases monotonically with problem size; however, the growth rate differs substantially.

The scalar implementation exhibits rapidly increasing execution times as the domain size grows, exceeding 140 seconds for the largest problem size (800×800). This behavior indicates poor cache reuse and increasing pressure on the memory subsystem as the working set exceeds cache capacity. In contrast, the SIMD-enabled version scales significantly more favorably. Although execution time still increases with domain size, the slope of the curve is much smaller. At 800×800 , the SIMD implementation completes in approximately 23 seconds, corresponding to a reduction in runtime of more than a factor of six relative to the scalar baseline. This widening performance gap demonstrates that SIMD parallelism becomes increasingly effective for larger problem sizes, where loop overhead and vectorization setup costs are amortized over a larger number of lattice nodes.

The near-linear growth of the SIMD curve further suggests improved utilization of vector units and more efficient streaming of data through the memory hierarchy, confirming the positive impact of the

data layout refactoring, branch elimination, and aligned memory accesses.

4.2.3. Speedup Analysis

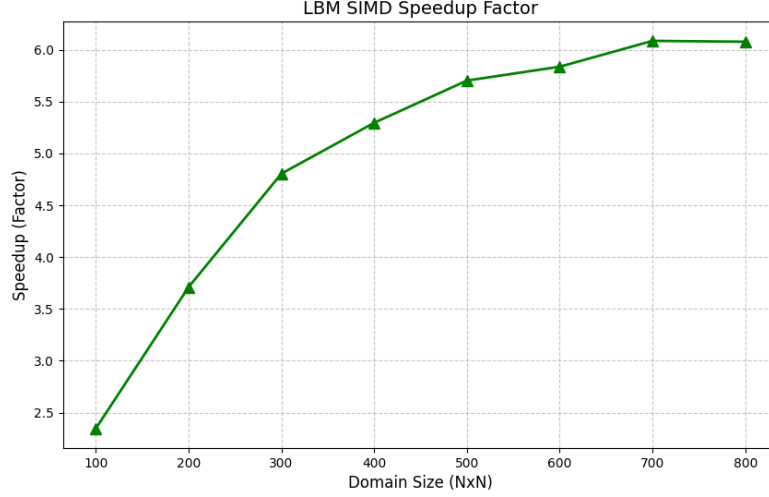


Figure 4.2.: Strong scaling: SIMD speedup factor relative to scalar baseline

Figure 4.2 shows the speedup achieved by the SIMD-enabled implementation relative to the scalar baseline. The speedup increases rapidly from approximately $2.3\times$ at 100×100 to nearly $5\times$ at 300×300 , indicating that vectorization overhead dominates for very small problem sizes but becomes negligible as the computational workload increases.

For larger domains, the speedup continues to rise more gradually and eventually saturates at approximately $6.1\times$ for domain sizes of 700×700 and above. This saturation behavior indicates a transition from a compute-bound regime to a memory-bandwidth-limited regime. Although wide SIMD execution is available, the achievable speedup is constrained by limited memory bandwidth, residual scalar instructions, and incomplete utilization of AVX-512 vector lanes due to gather operations and remaining control-flow complexity.

The observed plateau below the theoretical maximum of $8\times$ (expected from full double-precision AVX-512 vectorization) is consistent with the profiling results, which show partial vector lane occupancy and limited use of AVX-512 instructions. Nevertheless, the sustained speedup of approximately $6\times$ demonstrates that SIMD optimization provides substantial and scalable performance improvements for the LBM solver.

4.3. Potential Performance Improvements

4.3.1. Eliminating Pointer Chasing in `stream()`

Original Push-Based Implementation. The original streaming step iterates over nodes and *pushes* values to neighboring nodes using pointer dereferencing:

```
*neighbour_ptr ← value.
```

This constitutes a scatter operation with irregular memory access patterns. Such pointer chasing:

- Prevents effective auto-vectorization.
- Leads to frequent cache misses and TLB pressure.
- Introduces write conflicts and inhibits memory coalescing.

Optimized Pull-Based Streaming. The streaming operation is reformulated as an index-based *pull* scheme:

$$f_i^{(t+1)} = \sum_{j \in \mathcal{N}(i)} f_j^{(t)}.$$

Each node gathers the required distributions from its neighbors using precomputed integer indices rather than raw pointers.

- Writes are now contiguous and deterministic.
- The loop becomes amenable to SIMD vectorization.
- Although the read phase may involve gather operations, modern SIMD architectures (AVX2, AVX-512) provide hardware-supported gather instructions with acceptable throughput.

This transformation dramatically improves memory regularity and unlocks compiler vectorization opportunities.

4.3.2. Removing Conditionals from Hot Loops

Branching in the Original Loop. The original implementation contains a per-node conditional:

```
if (neighbour != nullptr).
```

Such branches inside tight loops:

- Inhibit vectorization or force masked execution.
- Introduce branch mispredictions.
- Reduce instruction-level parallelism.

Ghost Node Technique. All invalid or out-of-domain neighbor references are redirected to a dedicated *ghost node* (dummy memory location). Formally:

$$\forall i, \quad \mathcal{N}(i) \neq \emptyset.$$

- All neighbor accesses become unconditional and valid.
- The branch is completely eliminated.
- The resulting loop is branch-free and fully vectorizable.

This technique is crucial for maintaining high SIMD lane occupancy and avoiding masked vector execution.

4.3.3. Memory Alignment

Problem Statement. Standard `std::vector` does not guarantee alignment beyond the natural alignment of the contained type (typically 8 or 16 bytes). Misaligned memory accesses:

- Prevent the use of aligned load/store instructions.
- Introduce additional micro-operations and latency.

Solution: Aligned Allocation. A custom allocator is employed to guarantee SIMD-friendly alignment:

$$\text{alignment} = \begin{cases} 32 \text{ bytes} & (\text{AVX2, 256-bit}) \\ 64 \text{ bytes} & (\text{AVX-512, 512-bit}). \end{cases}$$

This enables the compiler to emit aligned vector loads and stores, maximizing bandwidth utilization and reducing pipeline stalls.

4.3.4. Ping-Pong Buffering

Original Implementation. At the beginning of each collision step, the contents of `nextValues` are copied into `values`, incurring an $\mathcal{O}(N)$ memory copy per time step.

Optimized Implementation. The copy is replaced by pointer swapping:

```
std::swap(current_distributions, next_distributions).
```

- Time complexity reduced to $\mathcal{O}(1)$.
- Eliminates unnecessary memory traffic.
- Improves cache locality and reduces pressure on the memory subsystem.

Part I.

Appendix

.1. Final Version of Code:

Repository Link:

<https://github.com/HangLi996/parallel-LBM-solver>

List of Figures

1.1. Discrete lattice directions and corresponding distribution functions f_i at a lattice node.	1
1.2. Algorithmic flowchart of the Lattice Boltzmann Method.	2
2.1. Different domain decomposition strategies for MPI parallelism: vertical, horizontal, and two-dimensional partitioning.	5
2.2. Messenger-based communication abstraction.	5
2.3. Code structure and execution flow of the MPI-parallel LBM solver, highlighting the separation between initialization and calculation phases.	6
2.4. Strong scaling performance comparison for different domain decomposition strategies.	10
2.5. Strong scaling of computation time.	11
2.6. Strong scaling of initialization time.	11
2.7. Ratio of initialization time to total runtime.	11
2.8. Breakdown of strong scaling results into computation and initialization phases, and the corresponding initialization time ratio.	11
2.9. Weak scaling performance (total runtime).	12
2.10. Weak scaling performance (computation phase only).	12
2.11. Weak scaling results for a fixed local problem size of 25×25 per process.	12
2.12. Scalasca function-level time percentage for the weak scaling experiment, focusing on the computation phase.	13
2.13. Weak scaling efficiency as a function of the number of MPI processes.	14
3.1. Total computation time over different number of OpenMP threads	19
3.2. Stream computation time over different number of OpenMP threads	20
3.3. Collision computation time over different number of OpenMP threads	20
4.1. Strong scaling: execution time comparison between scalar and SIMD-enabled builds	23
4.2. Strong scaling: SIMD speedup factor relative to scalar baseline	24

List of Tables

3.1. Performance breakdown for different OpenMP thread counts (MPI processes = 4, domain size = 80×80 , iterations = 1000).	17
3.2. Performance results for different OpenMP thread counts (MPI processes = 4, domain size = 80×80 , iterations = 1000).	18

Bibliography